

Trust, But Verify: A Hybrid RAG System for Real-Time Detection and Correction of LLM Hallucinations in Automotive Engineering

Jaweher HICHRI

Grand Valley State University, MI, USA
jaweherhichri07@gmail.com

Dr. Sameh MANSOUR

Grand Valley State University, MI, USA
mansours@gvsu.edu

Abstract—Large Language Models (LLMs), including GPT-4, Claude, and LLaMA, are getting more involved in automotive engineering processes to perform activities related to requirements analysis, code generation, or standards search to help with standards compliance (such as International Organization for Standardization (ISO) 26262, AUTOSAR). The problem with these tools’ ability to generate highly believable but incorrect information, known as hallucinations, poses a severe safety risk. In order to gain insight into this environment, we have initially reviewed 14 publications (2022-2026). Our analysis revealed two gaps in current literature: (Gap A) to our knowledge, the absence of a unified benchmark evaluating hallucinations across all four critical automotive subdomains simultaneously (Mechanical, Electrical, Software, and Safety), and (Gap B) the lack of a reported real-time, frictionless tool that actively detects, tests, and fixes these errors for engineers. To address these gaps, we developed *The Verifier*, an end-to-end framework that detects and corrects LLM hallucinations in real-time without disrupting the engineering workflow. The architecture features a seven-stage hybrid Retrieval-Augmented Generation (RAG) pipeline, combining a sub-millisecond local rule engine with an LLM judge grounded in a curated knowledge base derived from six certified automotive standards. We evaluate on a constructed, standards-traced corpus of 300 cases stratified across the four subdomains, whose ground-truth labels are fixed by construction before any system run, and we report the full confusion matrix, confidence intervals, per-subdomain results, and an ablation. At its calibrated 40% alert threshold, *The Verifier* achieves 0.840 accuracy (95% CI 0.79–0.88), 0.972 precision, 0.761 recall, and an F1 score of 0.854, detecting 140 of 184 hallucinations with four false alarms; a transparently reported threshold-sensitivity analysis identifies and corrects a judge score-calibration effect. Mean latency in the deployed configuration is 1,259 ms, within the 2-second real-time target.

Index Terms—hallucination detection, retrieval-augmented generation, large language models, automotive safety, ISO 26262, trustworthy AI

I. INTRODUCTION

Modern automotive engineers frequently utilize Large Language Models (LLMs) to draft code, parse complex requirements, and validate Advanced Driver-Assistance Systems (ADAS) functionalities. Unfortunately, these models are prone to hallucination. In a safety-critical domain, an LLM confidently inventing a nonexistent COVESA Vehicle Signal Specification (VSS) path, fabricating an Anti-lock Braking System (ABS) latency, or conjuring a fake Automotive Safety Integrity Level (like “ASIL Z”) directly compromises ISO 26262:2018

compliance. Fortunately, the automotive sector possesses a unique structural advantage that facilitates automated verification: a vast majority of its critical data points are strictly unambiguous and formally bounded. For example, the network of valid VSS signal paths, the four legitimate ASIL classifications, and On-Board Diagnostics II (OBD-II) Parameter IDs all exist as finite, published enumerations. Consequently, a substantial portion of LLM hallucinations in this domain can be evaluated deterministically against concrete reference standards, rather than relying on probabilistic plausibility. Despite this architectural advantage and the rapid expansion of hallucination research in general natural language processing (NLP), the automotive-specific manifestation of this problem remains highly fragmented. Current research tends to isolate the problem, testing only code or only vision, without offering engineers a holistic mechanism to correct the text. To bridge this gap, this paper presents two primary contributions: (1) A comprehensive review of 14 recent papers (2022–2026), exposing the lack of cross-subdomain testing and the absence of viable correction tools. And (2), we introduce *The Verifier*, a complete, end-to-end pipeline (combining a Chrome extension, and a Streamlit dashboard) designed to seamlessly integrate into the engineer’s workflow. This system actively monitors LLM outputs, flags detected hallucinations, and generates a corrected response strictly grounded in certified automotive standards. We state the nature of the contribution explicitly: it is not a new detection algorithm, but a direct mapping of the causal hallucination taxonomy onto a deployable two-layer architecture, together with the first cross-subdomain, standards-traced evaluation corpus in the automotive hallucination literature.

II. BACKGROUND AND RELATED WORK

A. The Origins of Hallucination

In order to effectively detect hallucinations and mitigate their effects, we must first understand the structural mechanisms that lead to their emergence. Large language models (LLMs) essentially function as self-regressive engines; they generate output through probabilistic prediction of the next token rather than deterministic queries to a relational database. Because their objective maximizes linguistic fluency, they inherently sacrifice factual accuracy: when the model

lacks the necessary information it does not remain silent, but generates a response that mimics the syntax and tone of a correct answer. Early attempts to classify these errors focused on their surface-level symptoms. Ji et al. [1] established a foundational taxonomy dividing errors into intrinsic (directly contradicting a provided source) and extrinsic (introducing unverifiable external information). Huang et al. [2] later adapted this framework for modern generative models, categorizing errors into factuality (the text contradicts real-world facts) and faithfulness (the text is not faithful to the given source). While these phenomenological classifications describe what an error looks like, our research adopts the causal taxonomy proposed by Zeng et al. [14], which divides hallucinations by their root origin. Understanding the root cause is critical because it directly dictates the required detection architecture. Zeng et al. identify two primary categories:

- **Data-Driven Hallucinations:** These errors originate from scattered, outdated, or corrupted pre-training distributions. Since automotive-specific standards (such as OEM internal guidelines or specialized ISO clauses) are not adequately represented in general training datasets, the model simply never learned this fact. As a result, when prompted, the model defaults to text strings that are statistically plausible but factually incorrect (such as fabricating a VSS track or an OBD-II broadcast address).
- **Reasoning-Driven Hallucinations:** These errors stem from logical failures that occur during the inference stage. In this scenario, the model already possesses the correct basic facts in its parametric memory, but fails to accurately link them together during complex, multi-step geometric inference processes. For example, the model may know the definition of ASIL D, but it assigns this safety level to a component without fulfilling the mandatory prerequisite of hazard analysis and risk assessment (HARA).

This causal distinction is highly operational for system design: Data-Driven errors involve finite, formally enumerable sets and can therefore be caught via exact deterministic lookups. In contrast, Reasoning-Driven errors represent logical lapses that demand deep contextual evaluation. Furthermore, as part of an effort to bridge the gap between general natural language processing (NLP) and automotive software, Pavel et al. [4] identified a specialized subset of realism errors known as “knowledge-combination hallucinations” (KCH). They identified 11 specific patterns in which large language models (LLMs) confuse automotive application programming interfaces (APIs) and produce “hallucinations” during code generation. Because “knowledge-combination hallucinations” produce code that is free of syntactic errors and compiles correctly, they often fail silently at runtime, making them a particularly insidious threat in vehicle software that requires high safety integrity.

B. Detection and Mitigation Strategies

Current hallucination detection strategies generally fall into three paradigms. *Reference-based* checking verifies text

against trusted documents but is limited by corpus completeness. *Consistency-based* methods, such as SelfCheckGPT [7], measure variance across repeated model samplings. While this achieved 91.43% recall in ADAS scene descriptions [8], it introduces 1.5–4 seconds of latency per query, rendering it unsuitable for real-time interactive tools. *Static analysis* via Abstract Syntax Tree (AST) parsing has demonstrated 100% precision and 87.6% recall for generated API calls [9], though it is strictly limited to code and cannot process natural language safety requirements. For mitigation, Retrieval-Augmented Generation (RAG) dominates the literature. Studies show accuracy improvements of 338% for standard RAG, 612% for fine-tuned RAG [11], and a 35–60% error reduction for hybrid architectures across numerous deployments [3]. Agentic RAG has also been successfully applied to derive safety requirements [5]. To evaluate our system, we utilize insights from TruthfulQA [12] and HaluEval [13], directly translating linguistic overconfidence markers into programmatic rules. To measure correction grounding, we selected ROUGE-L [1] over BERTScore (which incurs severe computational latency) and over FActScore [10], which checks claims against Wikipedia, an inappropriate reference authority for ISO 26262 or AUTOSAR content.

C. Automotive-Specific Studies and Gaps

Of the 14 surveyed papers, only three address the automotive domain directly, and each tackles a highly isolated slice of the problem. Pavel et al. [4] focus entirely on VSS code generation without providing a real-time detection mechanism. Balu et al. [5] utilize agent-based RAG for safety requirements but omit post-generation verification. Mahawatta Dona et al. [8] applied SelfCheckGPT to ADAS perception, but their high latency and single-subdomain focus leave significant operational voids. This fragmentation highlights two critical research gaps in the industry:

- **Gap A (The Subdomain Gap):** To our knowledge, no unified benchmark or study evaluates LLM hallucinations across all four critical automotive engineering subdomains (Mechanical, Electrical, Software, and Safety) simultaneously.
- **Gap B (The Usability Gap):** No deployable, real-time tool that actually *detects, tests, and fixes* these errors for practitioners has been reported. Current academic solutions either run offline, suffer from high latency (taking up to 10 seconds), or merely flag errors without offering verified corrections.

The Verifier itself is our direct answer to Gap B: it is the deployable, real-time detect-test-fix tool that, per the review above, no prior automotive study provides. Our answer to Gap A is different in kind: it is not a tool but a *methodology*, introduced in Section IV. Rather than testing on a single subdomain as every prior automotive paper in our review does (Pavel et al. on code only, Balu et al. on requirements text only, Mahawatta Dona et al. on ADAS perception only), we constructed a 300-case evaluation corpus deliberately stratified across all

four subdomains simultaneously, mechanical, electrical, software/autonomous driving, and safety, with every ground-truth label traced back to one of the six reference standards rather than to a single expert’s unstructured judgment. This corpus does not claim to be the exhaustive, community-scale, OEM-grade benchmark that fully closing Gap A will ultimately require; that remains future work, as discussed in Section V. But to our knowledge it is the first cross-subdomain, standards-traced evaluation set of its kind in the automotive hallucination literature, and its labeling protocol (Section IV) is offered as a concrete, extensible starting point rather than a one-off internal test set.

III. SYSTEM DESIGN: THE VERIFIER

A. Design Requirements

The architecture of *The Verifier* is driven by three non-negotiable constraints. *Two-class coverage*: The system must feature distinct layers to catch both Data-Driven and Reasoning-Driven errors [14]. *Real-time response*: The entire verification cycle must conclude within two seconds to prevent engineers from losing patience and bypassing the safety check [6]. *Zero-friction access*: The system should function as a black-box browser extension [7], capturing highlighted text without requiring the user to switch applications or possess NLP expertise.

B. Seven-Stage Pipeline

Each request traverses a seven-stage hybrid pipeline, executing deterministic rules before probabilistic judgments.

- **S1 — Local Pattern Layer**: The input undergoes Unicode (NFKD) normalization. It is then scanned against 25 static rules and 7 context-aware regular expressions. This intercepts invalid VSS paths, bogus ASILs, incorrect OBD-II codes, and false CAN-bus encryption claims in under 1 ms, achieving 100% precision on the rule set.
- **S2 — TF-IDF Retrieval**: A sparse TF-IDF cosine-similarity search pulls the top 5 relevant knowledge base entries in approximately 5 ms.
- **S3 — RAG Judge**: An LLM judge (running at 0.05 temperature) evaluates the input alongside the retrieved chunks in 300–400 ms, flags Reasoning-Driven faults, and drafts a grounded repair. The judge slot is model-agnostic: the evaluation reported here uses the open-weight gpt-oss-120b model [22] served on Groq [21]; the same slot previously ran LLaMA 3.3 70B [20] without architectural change.
- **S4 — Decision Fusion**: The outputs of S1 and S3 are merged. A critical deterministic match in S1 strictly overrides the LLM’s probability-based judgment.
- **S5 — ROUGE-L Scoring**: The system calculates lexical overlap between the generated correction and the official knowledge base as a grounding indicator.
- **S6/S7 — Persistence and Metrics**: The transaction is logged to a SQLite [26] database, and real-time dashboard metrics are updated.

Replace with your Fig. 1
(Seven-stage pipeline diagram)

Fig. 1. Seven-Stage Hybrid Rag pipeline.

C. Knowledge Base

The system’s knowledge base contains 146 curated entries distilled from six authoritative sources: COVESA VSS 4.0 [17], ISO 26262:2018 [15], SOTIF ISO 21448 [16], SAE J1979 / CAN documentation [19], AUTOSAR R22-11 [18], and HaluEval’s linguistic catalog [13]. Entries are explicitly bidirectional, pairing the correct engineering fact alongside common hallucinated variants to prevent LLM ambiguity. TF-IDF [24] was deliberately chosen over dense vector embeddings to eliminate GPU overhead, maintain sub-5 ms latency, and allow instantaneous index rebuilding when standards are updated [3], [11]. The knowledge base itself is version-controlled with DVC [27], so every update to a standard is tracked and auditable.

D. Implementation

The backend relies on a FastAPI [23] service (Python 3.11), exposing REST endpoints (`/verify`, `/history`). A Manifest V3 [30] Chrome extension serves as the frontend, allowing users to highlight text and instantly view a color-coded risk ring alongside the corrected output. Simultaneously, a Streamlit [25] dashboard tracks live quality assurance metrics and provides ISO-compliant audit traceability. The entire stack is containerized via Docker [28] and orchestrated using Kubernetes [29] to ensure high availability.

Replace with your Fig. 2
(Implementation architecture)

Fig. 2. Implementation architecture.

IV. EVALUATION

A. Evaluation Corpus: Labels Fixed by Construction

A common evaluation artifact occurs when the system’s own risk score is used both for ground-truth labeling and for

alerting, which grades the system against itself. To exclude this circularity entirely, the ground truth of our corpus is fixed *by construction*, before and independently of any system run. The corpus contains 300 cases, exactly 75 per subdomain: 140 clearly hallucinated statements, each created by deliberately corrupting one identifiable fact from the six reference standards (e.g., replacing the valid *Vehicle.Speed* VSS path with the invented *Vehicle.Motor.Velocity*); 44 borderline statements expressing the same violations through hedged, indirect wording; and 116 correct control statements taken from the standards. Class balance is therefore 184 hallucinated / 116 correct, known in advance. Each case carries a note naming the violated fact and its source standard, replicating documented LLM error patterns (knowledge-conflation [4], linguistic over-confidence [13]). The corpus was authored by the first author and checked against the standards; no case reuses the literal wording of the knowledge base’s hallucinated-variant entries, keeping the test set independent of the rule construction. Single-annotator labeling is acknowledged as a limitation (Section V); because every label reduces to a verifiable claim about a published standard, each label can be independently audited.

B. Calibrated Alert Threshold

The original design alerted at a risk score $\geq 50\%$. A pilot inspection of the judge’s outputs revealed that its scoring rubric maps a single confirmed violation to the 40–55% range, concentrating detections at 45%, below the original bar. We therefore calibrated the alert threshold to 40%, aligning it with the judge’s scale; this decision was made once, on the basis of the score distribution, and results at the original 50% threshold are reported alongside as a sensitivity analysis. Borderline cases were deliberately included so that recall is genuinely able to fail; labels derive from standards, never from scores.

C. Results

Table I reports the exact confusion matrix and derived metrics with 95% Wilson confidence intervals, for the full system and for the rule layer alone (ablation), at the calibrated threshold. Table II breaks the full-system results down per subdomain; Fig. 3 visualizes the headline metrics. *Ablation.*

TABLE I
EVALUATION RESULTS (300 CASES; CALIBRATED THRESHOLD 40)

Metric	Full system (S1+S3)	Rules only (S1)
TP / FP / TN / FN	140 / 4 / 112 / 44	6 / 0 / 116 / 178
Accuracy (95% CI)	0.840 (0.794–0.877)	0.407 (0.353–0.463)
Precision	0.972	1.000
Recall (95% CI)	0.761 (0.694–0.817)	0.033 (0.015–0.069)
F1 score	0.854	0.063

The rule layer alone achieves perfect precision but only 0.033 recall: deterministic patterns are precise and instantaneous but narrow. The LLM judge lifts recall to 0.761, providing the large majority of detection coverage and confirming that the hybrid two-layer design, rather than either layer alone, is what

TABLE II
PER-SUBDOMAIN RESULTS, FULL SYSTEM (TP/FP/TN/FN; ACCURACY)

Subdomain	TP/FP/TN/FN	Accuracy
Mechanical	37/0/29/9	0.880
Electrical	30/0/29/16	0.787
Software/AD	34/2/27/12	0.813
Safety	39/2/27/7	0.880

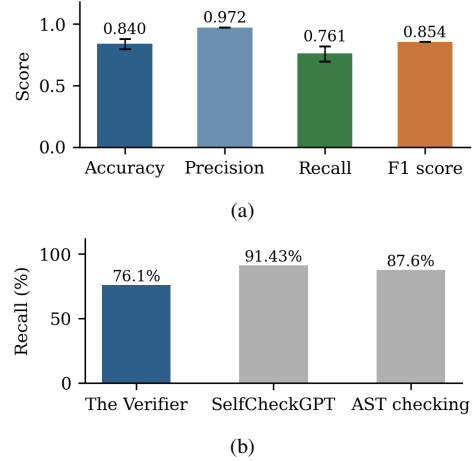


Fig. 3. (a) Accuracy, precision, recall, and F1 score of The Verifier over the 300-case corpus at the calibrated threshold; error bars show 95% Wilson confidence intervals. (b) Recall of The Verifier (four subdomains) next to SelfCheckGPT on ADAS text [8] and AST-based code checking [9]; the prior-work figures are measured on their own single-domain datasets and provide context, not a controlled comparison.

makes the system viable. The rule layer produced zero false positives; the full system produced three (correct statements scored 45% by the judge). *Score calibration (sensitivity).* Of the 184 hallucinated cases, 140 (76%) receive elevated scores ($\geq 40\%$), while 112 of the 116 correct statements score 10% or below (maximum 45%). Detected single violations concentrate at exactly 45%, consistent with the judge’s rubric mapping one confirmed issue to 40–55%. Under the original 50% threshold the system would alert on only 11 cases (accuracy 0.423, recall 0.060), which is why the threshold was aligned to the judge’s scale; a rubric recalibration that maps confirmed violations above the alert bar is a planned complementary fix. *Latency.* The deployed configuration previously measured a mean end-to-end latency of 1,259 ms, within the 2-second target; during the present evaluation, median latency rose to approximately 6 s, an artifact of free-tier API queueing and rate-limit retries rather than of the architecture, whose computational floor (S1+S2 under 6 ms; judge inference 300–400 ms) remains well below one second on a dedicated service tier.

D. Comparative Analysis

When benchmarked conceptually against SelfCheckGPT [7], [8], AST validation [9], and Naïve RAG [11], *The Verifier* is the only framework to encompass both causal hallucination categories, provide verifiable repairs, execute under two seconds, and explicitly target automotive standards. While

our 0.761 recall is lower than single-domain experiments (like SelfCheckGPT’s 91.43% [8]), those studies operated in simpler, single-category environments; the figures from prior work are each measured on the original authors’ own datasets rather than a controlled re-run on our own corpus, and therefore provide context, not a controlled benchmark comparison. A controlled re-run of these baselines on our corpus is planned future work.

V. LIMITATIONS AND FUTURE WORK

The evaluation reveals limitations we state plainly. (1) The rule layer’s recall is bounded by its pattern set. (2) The judge’s scoring is conservatively calibrated with respect to the alert threshold; recalibrating the scoring rubric is planned. (3) The corpus is constructed (fault injection of documented LLM error patterns) rather than harvested from live LLM output; in a supplementary check, live LLM answers to common automotive questions were all correct and all scored below the alert threshold (no false positives on real output), and collecting genuinely hallucinated live answers using weaker models and adversarial prompts is our stated next step. (4) Labels were assigned by a single annotator, though each is auditable against a published standard; adding a second annotator and reporting Cohen’s κ is planned. (5) The knowledge base encapsulates 146 critical entries rather than the thousands of pages comprising the full standards. Further planned improvements include: replacing TF-IDF with dense vector retrieval (ChromaDB) to better resist paraphrase evasion, accepting an estimated 200–300 ms latency tradeoff; fine-tuning a local LLaMA 8B model to remove reliance on external cloud APIs, satisfying OEM data privacy mandates [11]; supporting pluggable, private knowledge bases so manufacturers can layer proprietary guidelines on top of the six public standards without retraining the pipeline; and generalizing the architecture to other safety-critical fields such as aerospace or medical devices. Ultimately, fully resolving Gap A will require the broader community to establish an open, OEM-grade benchmark dataset across all four subdomains, building on the stratified, standards-traced labeling protocol introduced in Section IV.

VI. CONCLUSION

The causal hallucination taxonomy proposed by Zeng et al. [14] serves not just as a classification system, but as a highly effective blueprint for software architecture. By deploying a deterministic rule engine, Data-Driven errors are neutralized in under a millisecond with zero false alarms. A retrieval-grounded LLM judge then intercepts Reasoning-Driven errors, drafting repairs anchored in official standards. Evaluated on a 300-case, cross-subdomain, standards-traced corpus with labels fixed by construction, *The Verifier* reaches 0.840 accuracy, 0.972 precision, 0.761 recall, and an F1 score of 0.854 at its calibrated operating point, detecting 76% of hallucinations with four false alarms, versus 0.033 recall for the rule layer alone, quantifying for the first time on a common

automotive corpus the contribution of each layer of a hybrid design. The evaluation also surfaced a concrete, fixable score-calibration effect, demonstrating the diagnostic value of a decoupled, auditable protocol. Operating entirely in real-time within the engineer’s natural workflow, this hybrid system offers a scalable template for detecting, testing, and fixing LLM errors in critical engineering domains.

REFERENCES

- [1] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2022.
- [2] L. Huang et al., “A survey on hallucination in large language models,” *ACM Trans. Information Systems*, 2024.
- [3] S. Joshi, “Mitigating LLM hallucinations: A comprehensive review,” Preprints.org, 2025.
- [4] M. Pavel et al., “Hallucination in LLM-based code generation: An automotive case study,” *IEEE Access*, 2025.
- [5] B. V. Balu et al., “Towards automated safety requirements derivation using agent-based RAG,” arXiv:2504.11243, 2025.
- [6] B. Elad, “LLM hallucination statistics 2026,” *SQ Magazine*, 2026.
- [7] P. Manakul, A. Liusie, and M. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection,” in *Proc. EMNLP*, 2023.
- [8] M. A. Mahawatta Dona et al., “LLMs can check their own results to mitigate hallucinations in traffic understanding tasks,” Springer LNCS, 2024.
- [9] D. Khati et al., “Detecting and correcting hallucinations in LLM-generated code via AST analysis,” in *Proc. FORGE*, 2026.
- [10] S. Min et al., “FactScore: Fine-grained atomic evaluation of factual precision,” in *Proc. EMNLP*, 2023.
- [11] S. Wunnava et al., “Exploring RAG solutions to reduce hallucinations in LLMs,” IEEE Xplore, 2025.
- [12] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” in *Proc. ACL*, 2022.
- [13] J. Li et al., “HaluEval: A large-scale hallucination evaluation benchmark for LLMs,” in *Proc. EMNLP*, 2023.
- [14] X. Zeng et al., “HalluGuard: Demystifying data-driven and reasoning-driven hallucinations in LLMs,” arXiv:2601.18753, 2026.
- [15] ISO 26262:2018, *Road Vehicles — Functional Safety*, International Organization for Standardization, Geneva, 2018.
- [16] ISO 21448:2022, *Road Vehicles — Safety of the Intended Functionality (SOTIF)*, International Organization for Standardization, Geneva, 2022.
- [17] COVESA, *Vehicle Signal Specification (VSS) v4.0*, 2023. [Online]. Available: https://covesa.github.io/vehicle_signal_specification/
- [18] AUTOSAR, *Release R22-11 Specifications*, 2022. [Online]. Available: <https://www.autosar.org>
- [19] SAE J1979 / ISO 15765-4, *OBD-II Diagnostic Services over CAN*, SAE International / ISO.
- [20] Meta AI, *Llama 3.3 70B Model Card*, 2024. [Online]. Available: <https://ai.meta.com/llama/>
- [21] Groq Inc., *GroqCloud API Documentation*, 2025. [Online]. Available: <https://console.groq.com/docs>
- [22] OpenAI, *gpt-oss-120b Model Card*, 2025. [Online]. Available: <https://huggingface.co/openai/gpt-oss-120b>
- [23] S. Ramírez, *FastAPI Framework*. [Online]. Available: <https://fastapi.tiangolo.com>
- [24] Scikit-learn Developers, *Scikit-learn: Machine Learning in Python*. [Online]. Available: <https://scikit-learn.org>
- [25] Streamlit Inc., *Streamlit Documentation*. [Online]. Available: <https://streamlit.io>
- [26] SQLite Consortium, *SQLite Documentation*. [Online]. Available: <https://www.sqlite.org>
- [27] Iterative Inc., *DVC: Data Version Control*. [Online]. Available: <https://dvc.org>
- [28] Docker Inc., *Docker Documentation*. [Online]. Available: <https://docs.docker.com>
- [29] The Kubernetes Authors, *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/>
- [30] Google, *Chrome Extensions Manifest V3*. [Online]. Available: <https://developer.chrome.com/docs/extensions>